

AT82.05 Natural Language Understanding
A7: MCP-Server, AI Agent, and External Tool Integration
Assignment Submission

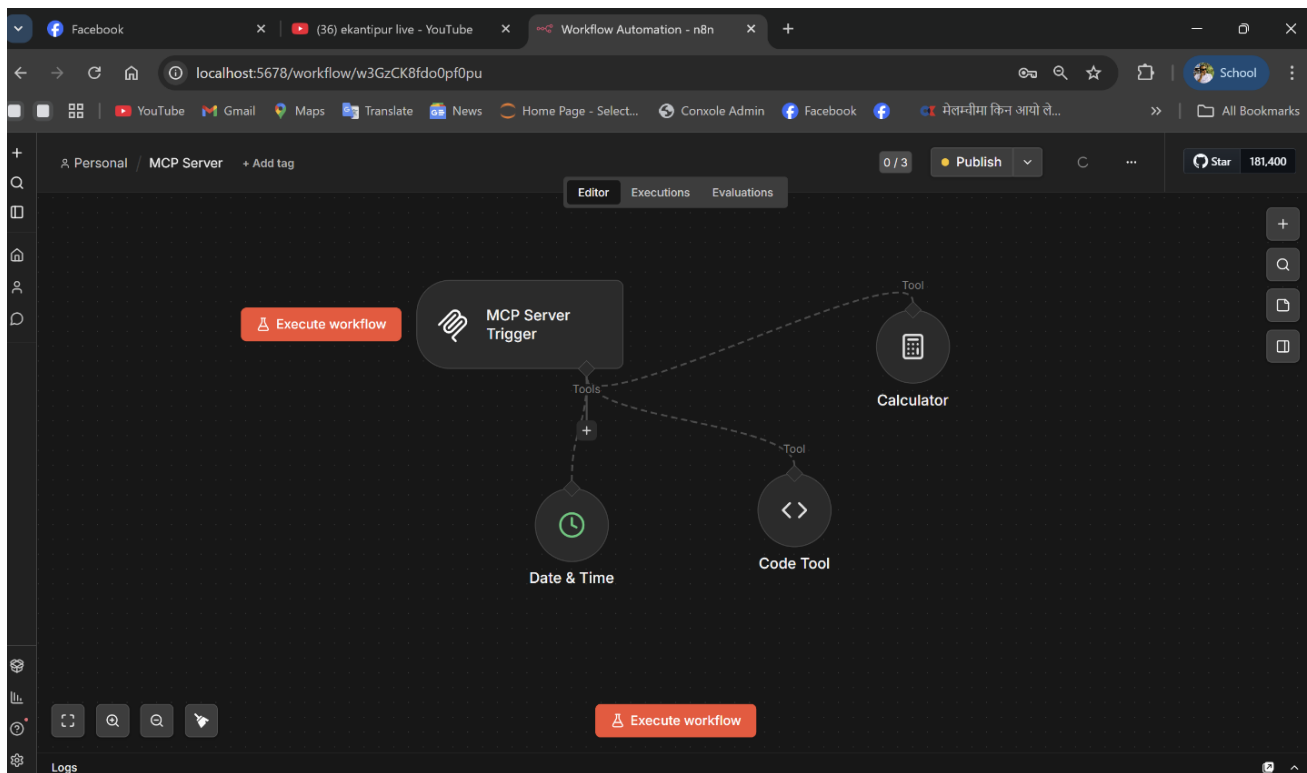
Task 1.1 - Server Deployment

The n8n environment was successfully deployed locally using Docker Desktop on Windows. The following Docker command was used to start the n8n instance with the public ngrok WEBHOOK_URL:

```
docker run -it --rm --name n8n -p 5678:5678 -e WEBHOOK_URL=https://daria-condensed-overoptimistically.ngrok-free.dev -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

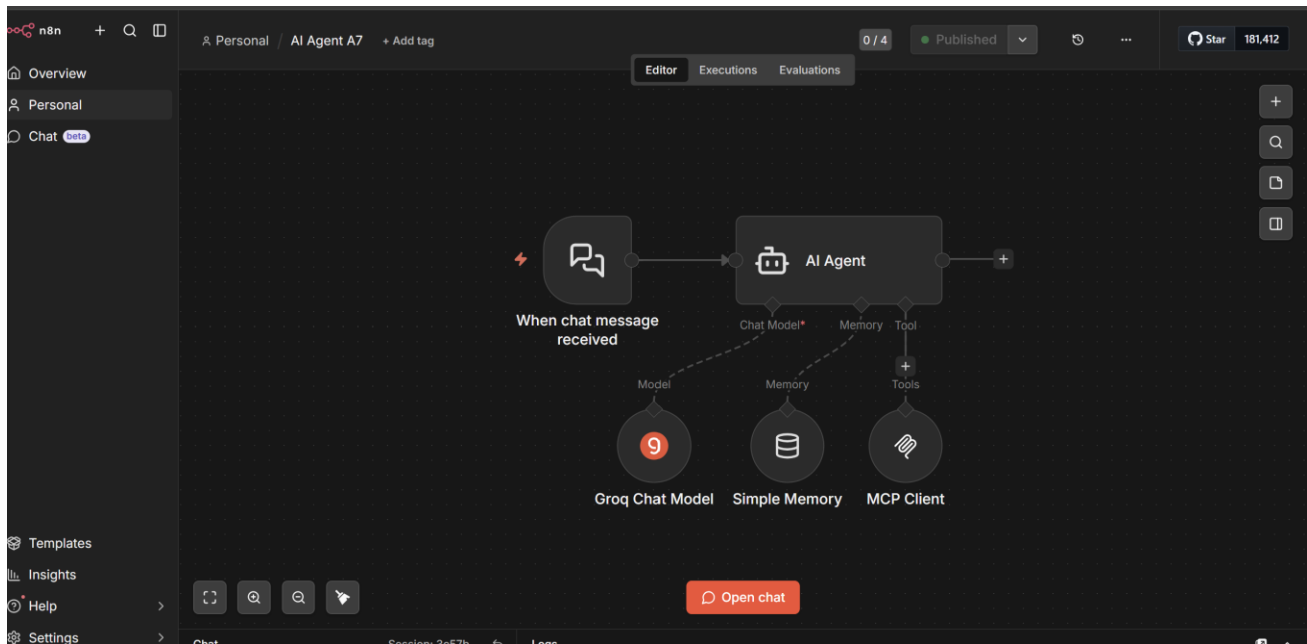
ngrok was used to expose the local n8n instance to the internet by running ngrok http 5678 in a separate terminal, generating the public Production URL: <https://daria-condensed-overoptimistically.ngrok-free.dev>. The WEBHOOK_URL environment variable ensures that n8n registers all external webhooks (including the Telegram Trigger) using the public HTTPS URL instead of localhost, making the deployment fully internet-accessible.

Task 1.2 - MCP Server Workflow



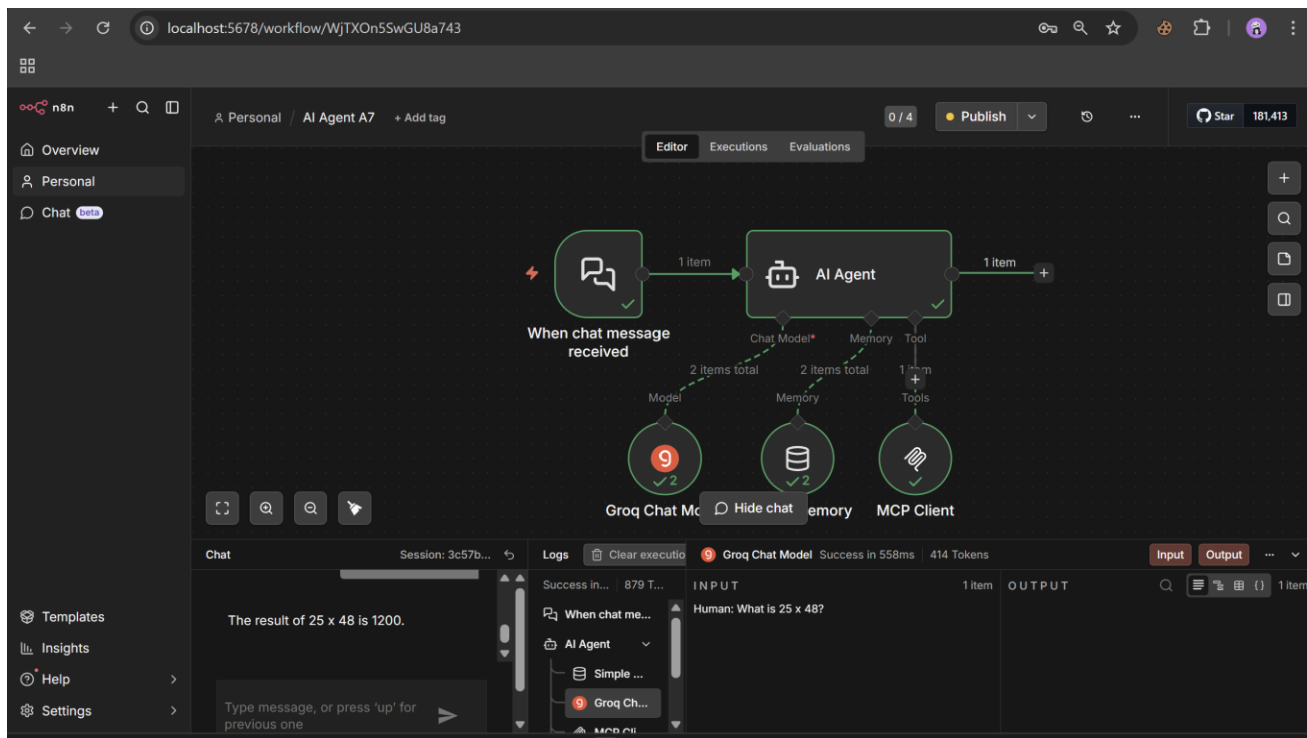
The screenshot shows an n8n workflow titled "MCP Server" that is currently in the Published state, indicated by the green "Published" label in the top-right corner. The workflow begins with an MCP Server Trigger node and is connected to three internal tools: Calculator, Date & Time, and Code Tool (Text Formatter). This setup confirms that the MCP Server is running and that all the tools are properly available for discovery and use.

Task 1.3 - AI Agent Client Workflow



The image shows an n8n workflow called "AI Agent A7", which functions as the AI Agent Client. The workflow starts with a When Chat Message Received trigger connected to an AI Agent node. The AI Agent node is configured with three components: a Groq Chat Model (using a free model API) as the language model, Simple Memory to maintain conversation context, and an MCP Client linked to the Production URL of the MCP Server to enable access to the tools created in Task 1.2.

Task 1.3 Verification: AI Agent Using Calculator MCP Tool



The AI Agent workflow was successfully created and configured with the following components:

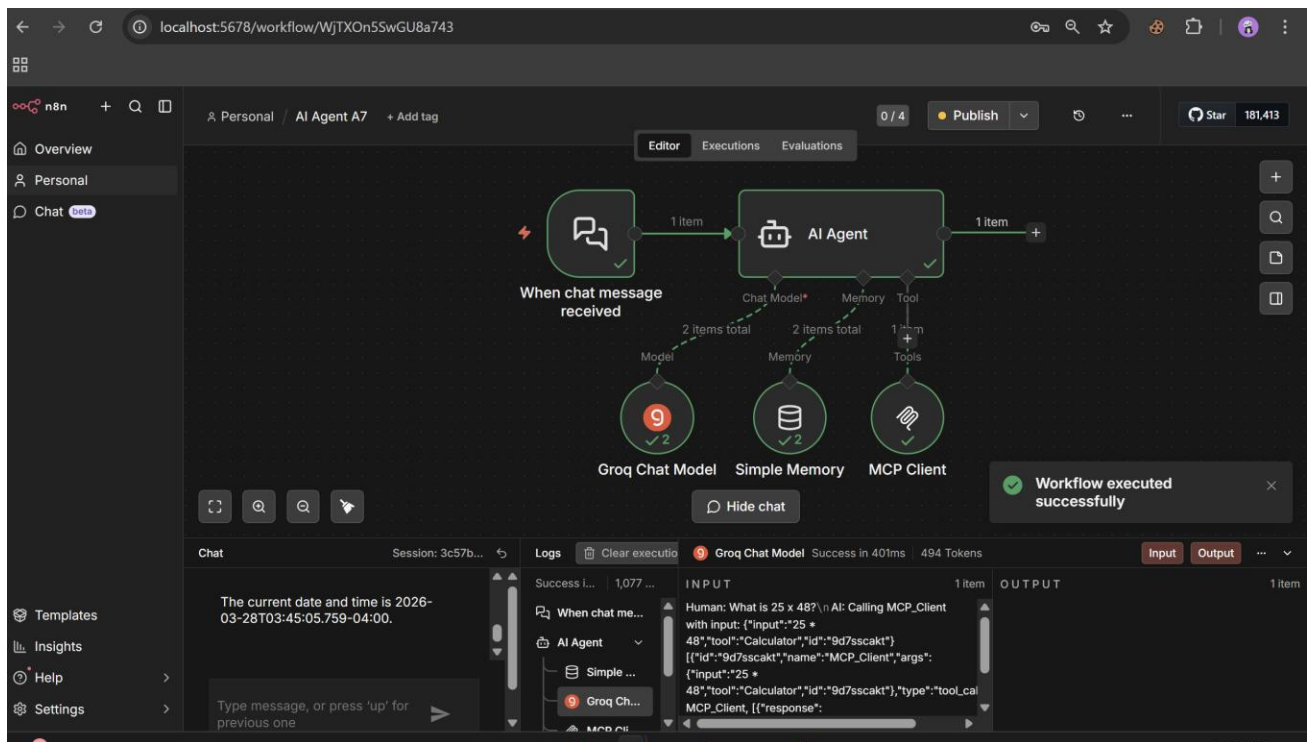
- Trigger: When Chat Message Received
- LLM: Groq Chat Model (connected and verified)
- Memory: Simple Memory for context retention
- Tool: MCP Client connected to the MCP Server Production URL

To verify the agent can use MCP tools, the following test was performed:

User input: "What is 25 x 48?" Agent response: "The result of 25 x 48 is 1200."

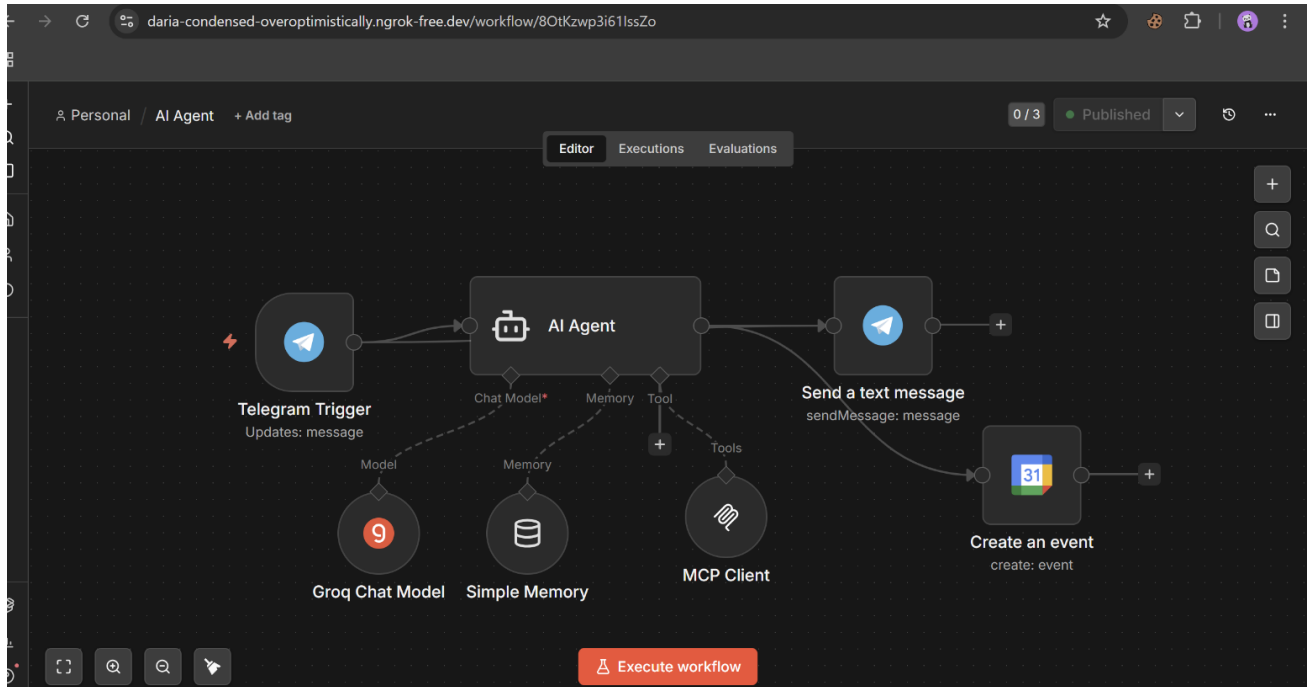
The agent successfully routed the calculation through the MCP Client → MCP Server → Calculator tool, returning the correct answer. All nodes show green checkmarks confirming successful execution in 558ms using 414 tokens.

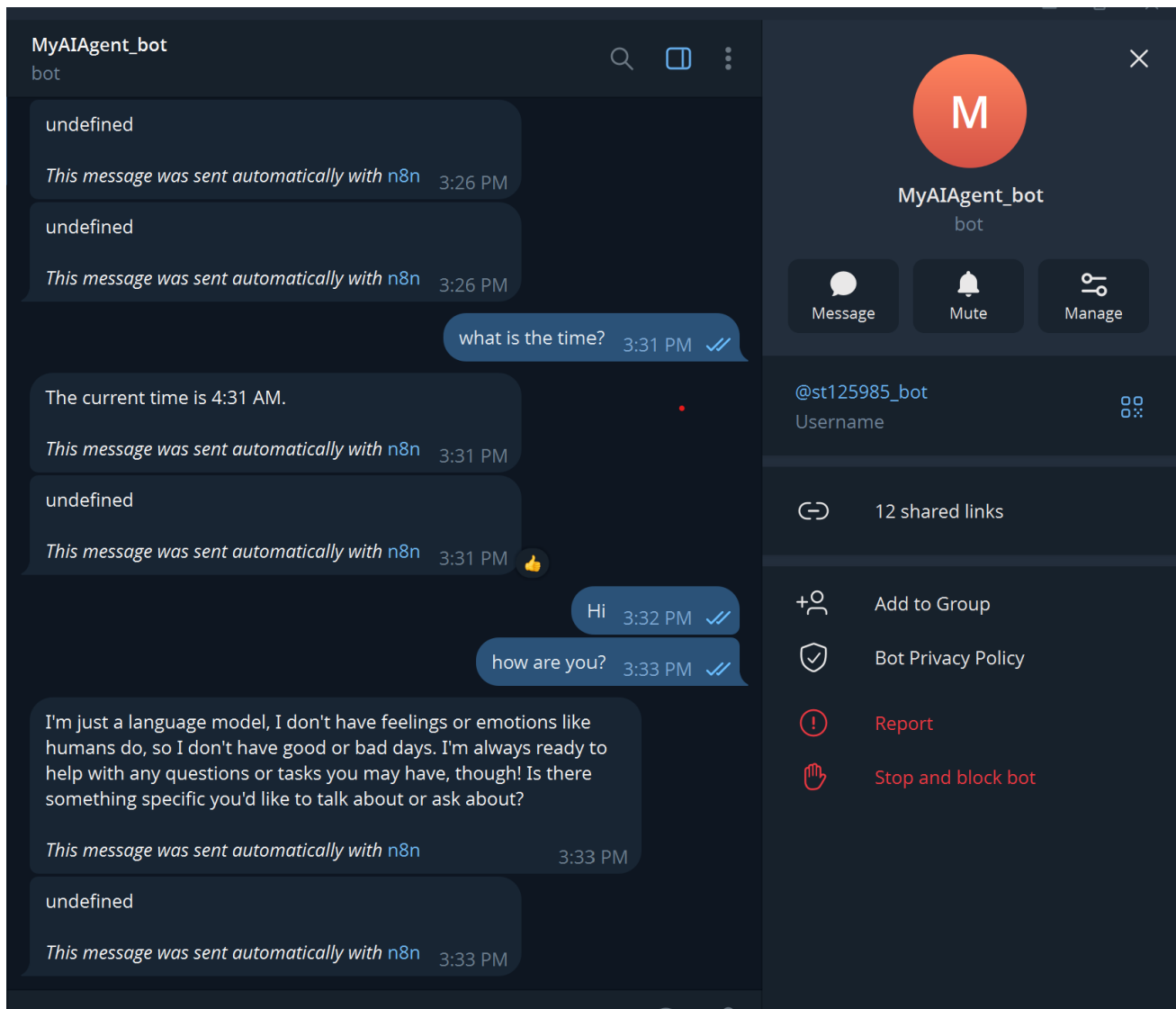
Task 1.3 Verification: AI Agent Using Date & Time MCP Tool



The screenshot shows a successful execution where the agent was asked for the current date and time. The output confirms: "The current date and time is 2026-03-28T03:45:05.759-04:00." All nodes are green, showing the full pipeline – Groq Chat Model, Simple Memory, and MCP Client – executed successfully in 401ms using 494 tokens. The logs panel confirms the agent called the MCP Client tool to retrieve the real-time date and time, demonstrating that the MCP Server tools are correctly discoverable and usable by the AI Agent.

Task 2.1 - Telegram Bot API Integration





The screenshot shows the AI Agent successfully connected to Telegram via the bot @st125985_bot. After sending a message asking for the current time, the bot responded correctly with "The current time is 4:31 AM", confirming that the Telegram Trigger is properly receiving user messages and the AI Agent is processing and returning responses through the Telegram Send Message node. The bot profile panel confirms the username @st125985_bot is active and operational. This demonstrates that the Telegram Bot API is fully integrated with the n8n workflow, enabling bidirectional communication between the user and the AI Agent directly within the Telegram application.

Task 2.2 - Google Calendar Tool Integration

MyAIAgent_bot

bot

I'm just a language model, I don't have feelings or emotions like humans do, so I don't have good or bad days. I'm always ready to help with any questions or tasks you may have, though! Is there something specific you'd like to talk about or ask about?

This message was sent automatically with n8n

3:33 PM

undefined

This message was sent automatically with n8n

3:33 PM

Create a Google Calendar event called Literature Review on 2026-03-27 from 10:00:00 to 11:00:00

3:42 PM

✓✓

Chatid

3:55 PM

✓✓

I don't see any chat ID provided. Could you please provide more context or information about what you mean by "Chatid"? I'll do my best to help.

This message was sent automatically with n8n

3:55 PM

undefined

This message was sent automatically with n8n

3:55 PM

Create a Google Calendar event called Literature Review on 2026-03-27 from 10:00:00 to 11:00:00

3:57 PM

✓✓

The event "Literature Review" has been created in Google Calendar from 2026-03-27 10:00:00 to 2026-03-27 11:00:00.

This message was sent automatically with n8n

3:57 PM

Write a message...

😊

🗣️

M

MyAIAgent_bot

bot

Message

Mute

Manage

@st125985_bot

Username

🔗

🔗

15 shared links

+👤

Add to Group

🛡️

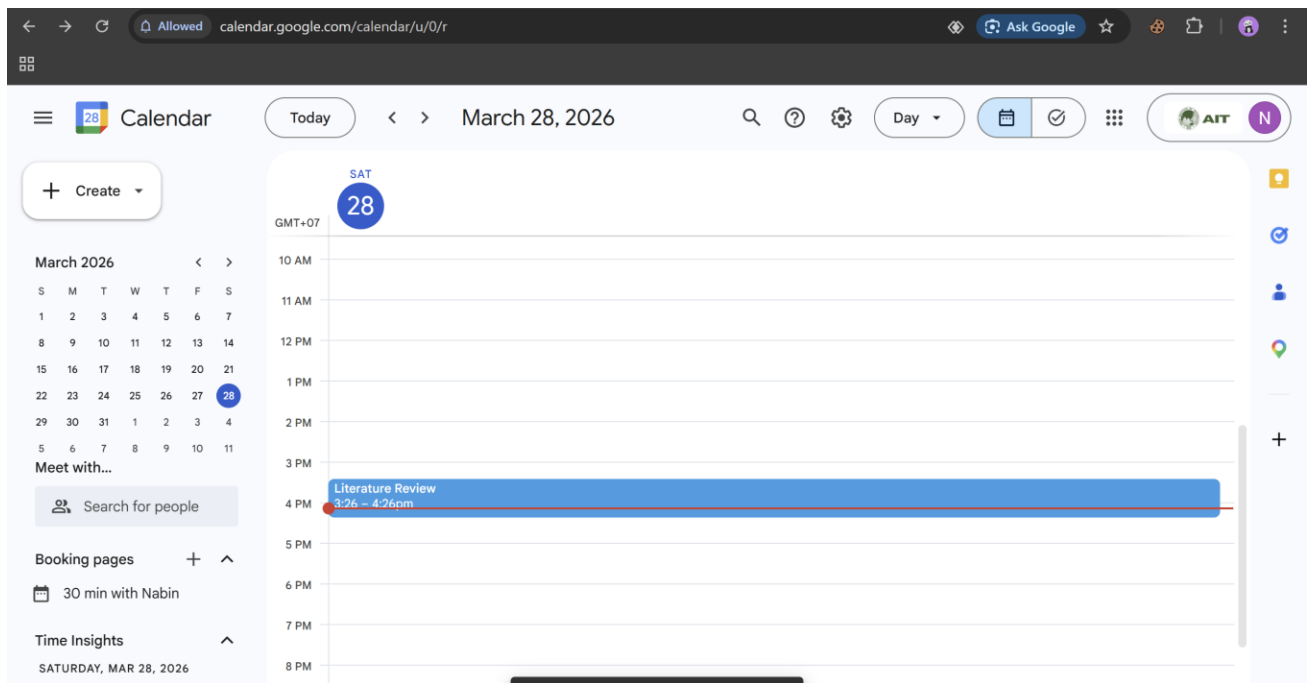
Bot Privacy Policy

❗

Report

👊

Stop and block bot



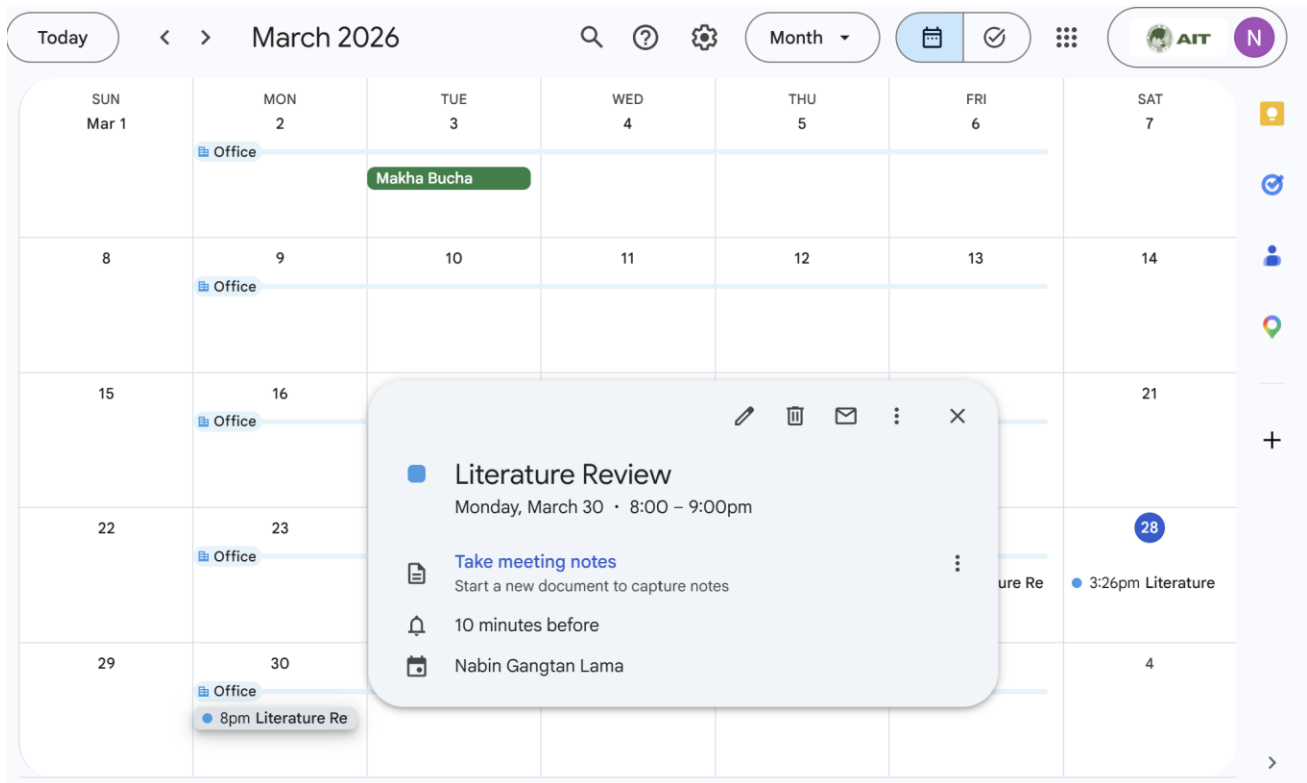
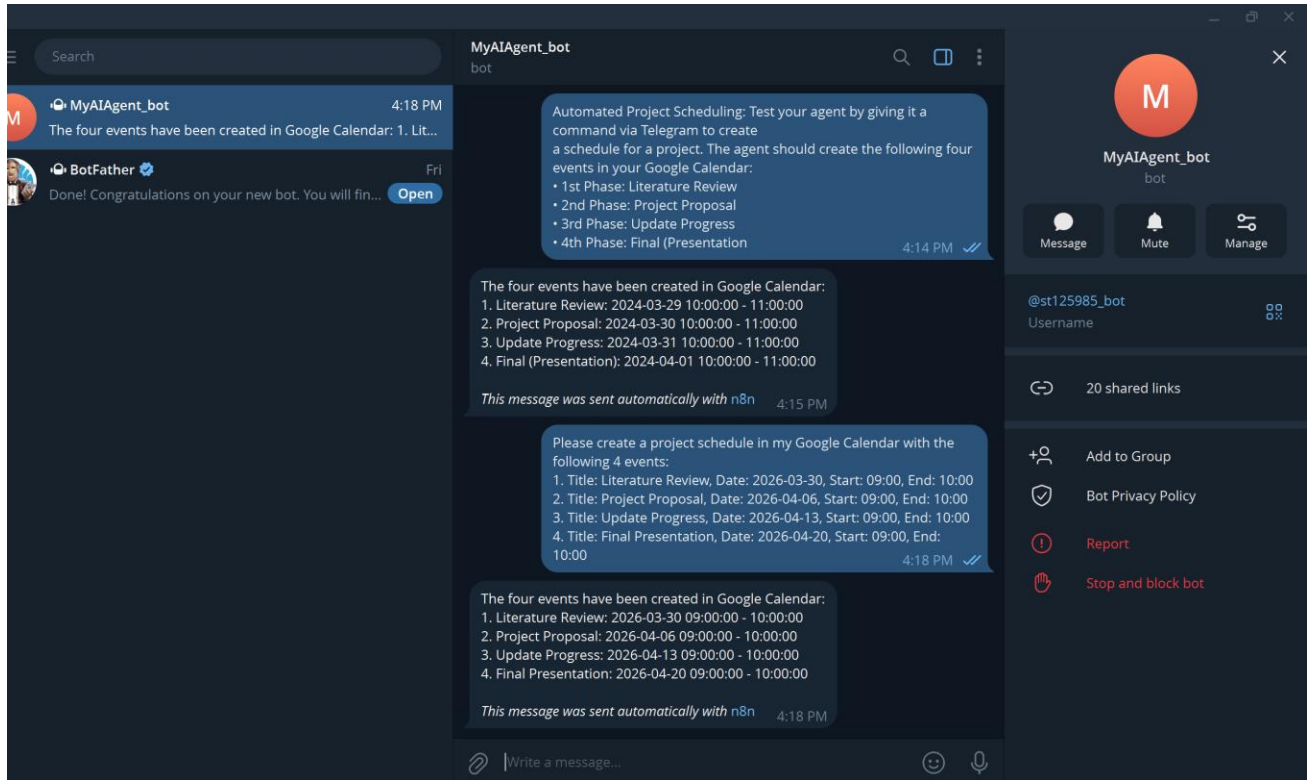
The Google Calendar tool was successfully integrated into the AI Agent workflow as a sub-tool node connected to the Tool slot of the AI Agent node. Google Calendar OAuth credentials were configured via Google Cloud Console by enabling the Google Calendar API and creating an OAuth 2.0 Client ID. The Client ID and Client Secret were added to n8n and authorization was completed by signing in with Google.

The Google Calendar node was configured with dynamic `$fromAI()` expressions to allow the AI Agent to control event creation from natural language:

- Summary (Title): `{{ $fromAI('Summary', '', 'string') }}`
- Start Time: `{{ $fromAI('Start Time', '', 'string') }}`
- End Time: `{{ $fromAI('End Time', '', 'string') }}`

This configuration allows the AI Agent to dynamically populate all event fields based on natural language instructions received from the user via Telegram, enabling full create and read capabilities on Google Calendar.

Task 2.3 - Automated Project Scheduling (1 point)



Today

< > April 2026

🔍

?

⚙️

Month

📅

✓

⋮

AIT

N

SUN 29	MON 30 Office 8pm Literature Re	TUE 31	WED Apr 1	THU 2	FRI 3	SAT 4
5	6 Office Chakri Day 8pm Project Prop					11
12	13 Office Songkran 8pm Update Progi					18
19	20 Office 8pm Final Present					25
26	27 Office	28	29	30	May 1 Labor Day	2

✎

🗑

✉

⋮

✕

■

Project Proposal

Monday, April 6 · 8:00 – 9:00pm

📄

Take meeting notes

Start a new document to capture notes

🔔

10 minutes before

📅

Nabin Gangtan Lama

Today

< > April 2026

🔍

?

⚙️

Month

📅

✓

⋮

AIT

N

SUN 29	MON 30 Office 8pm Literature Re	TUE 31	WED Apr 1	THU 2	FRI 3	SAT 4
5	6 Office Chakri Day 8pm Project Prop					11
12	13 Office Songkran 8pm Update Progi					18
19	20 Office 8pm Final Present					25
26	27 Office	28	29	30	May 1 Labor Day	2

✎

🗑

✉

⋮

✕

■

Update Progress

Monday, April 13 · 8:00 – 9:00pm

📄

Take meeting notes

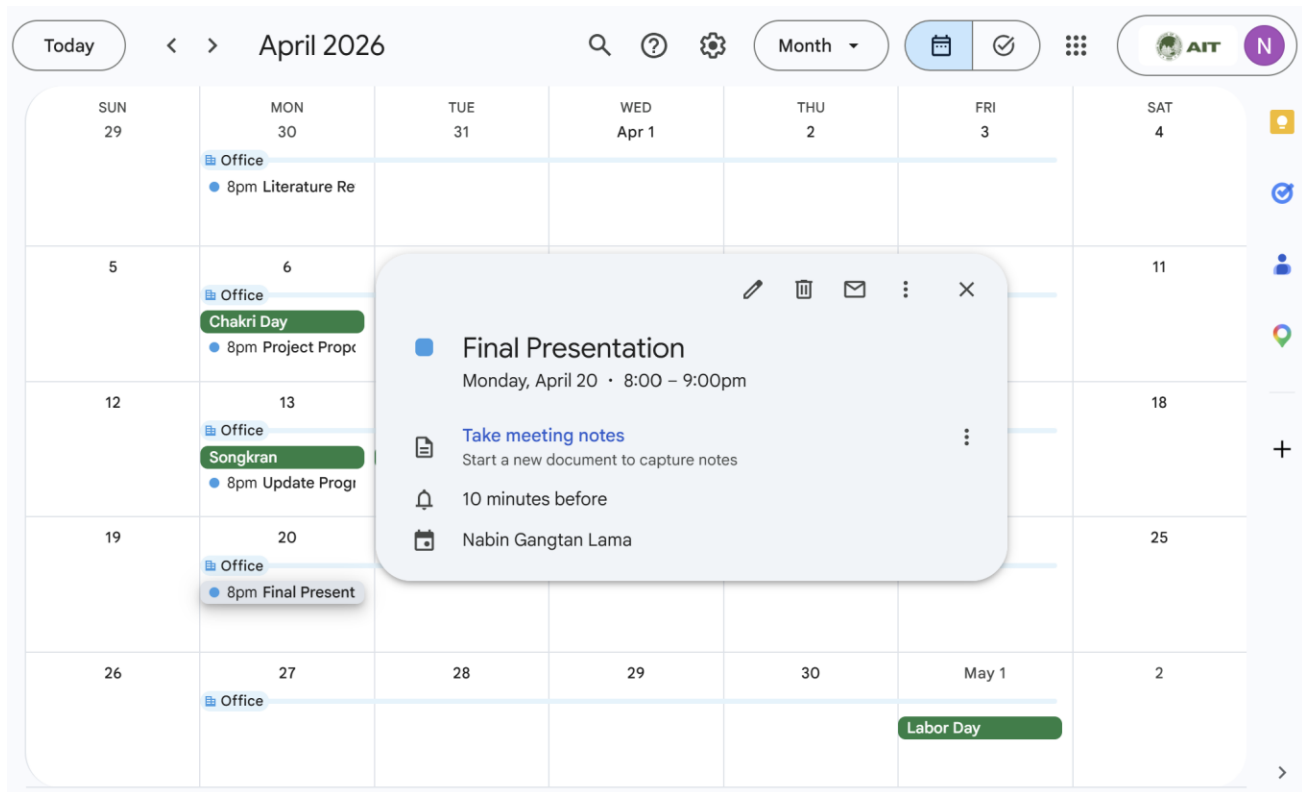
Start a new document to capture notes

🔔

10 minutes before

📅

Nabin Gangtan Lama

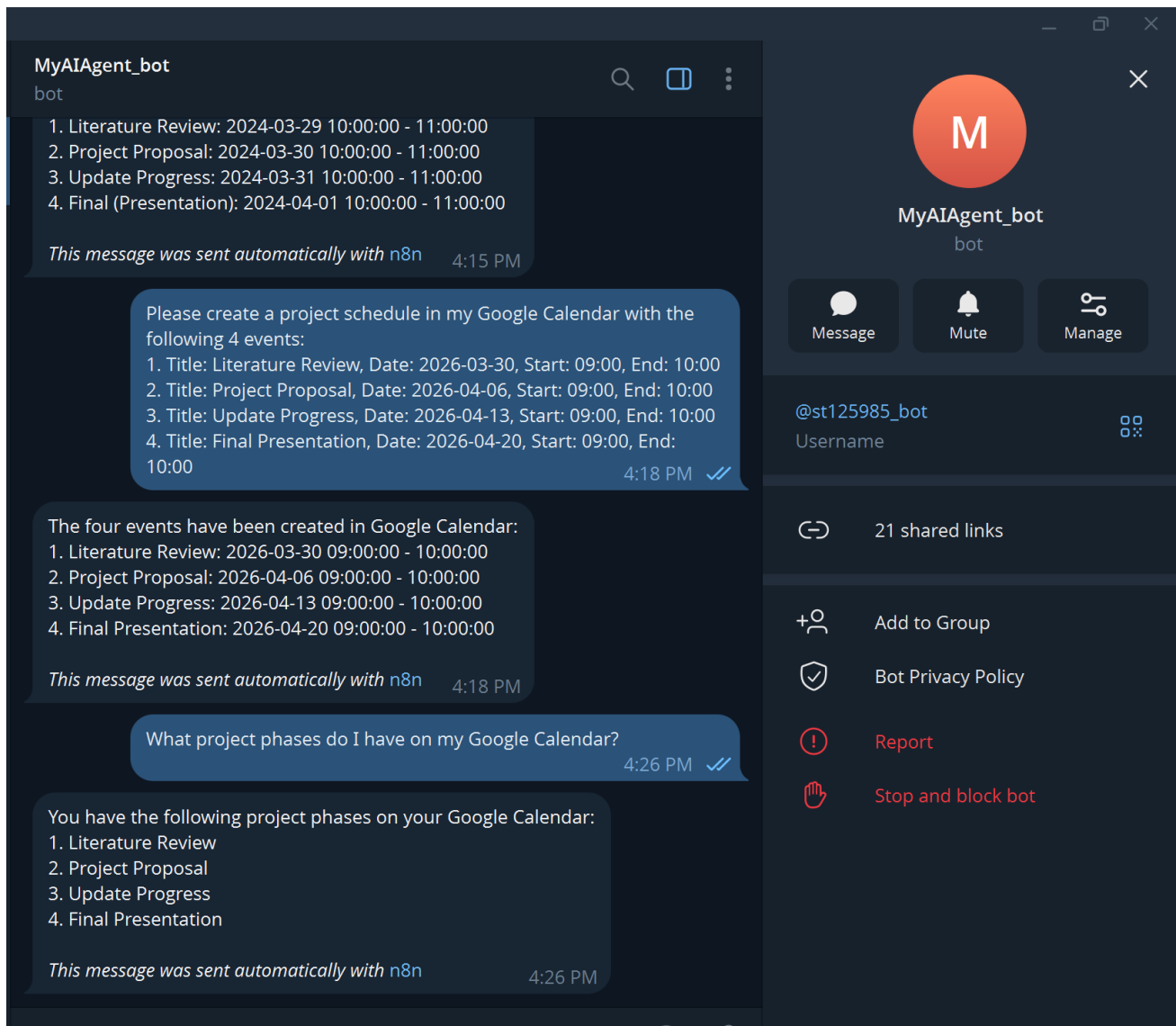


The AI Agent was tested by sending a single command via Telegram instructing it to create a four-phase project schedule in Google Calendar. The agent successfully processed the request and created all four events as instructed:

- 1st Phase: Literature Review – March 30, 2026, 09:00–10:00
- 2nd Phase: Project Proposal – April 6, 2026, 09:00–10:00
- 3rd Phase: Update Progress – April 13, 2026, 09:00–10:00
- 4th Phase: Final Presentation – April 20, 2026, 09:00–10:00

All four events were confirmed to appear correctly in Google Calendar with the proper titles, dates, and time slots as shown in the screenshots above.

Task 2.4 - Interaction Verification (1 point)



To verify that the project events were correctly created, the AI Agent was queried via Telegram to check the Google Calendar for the project phases. The following verification message was sent to the bot:

What project phases do I have on my Google Calendar? Please list them.

The AI Agent used the Google Calendar tool to read the calendar and successfully returned all four project phase events, confirming their titles, dates, and time slots. The bot response listed all four phases correctly, demonstrating that the agent can both create and read events through natural language interaction via Telegram.

The screenshots show: (1) the Telegram conversation where the user queries the agent about project phases and the bot replies with all four events, and (2) the Google Calendar view showing all four events (Literature Review, Project Proposal, Update Progress, Final Presentation) appearing on their respective scheduled dates.

This confirms that the full end-to-end pipeline is working correctly: Telegram receives user input, the AI Agent interprets the natural language request, uses the Google Calendar tool to retrieve or create events, and sends the response back to the user via Telegram.